

Chapter 12

Carrier Recovery

载波恢复

载波恢复

影响数字相干光系统的主要有两种相位效应：载波频率偏移和相位噪声。

1.载波频率偏移： 发射器和本地振荡器激光器的工作频率并不完全相同，但会出现偏移，偏移量可达几**Ghz**。

2.相位噪声： 在激光器中，除了所需的受激辐射外，光子的**自发辐射**还会产生时间连续的维纳过程相位噪声。

载波恢复

假定均衡和解偏都是完美的，受频率偏移、相位噪声和加性噪声影响的均衡信号 $\mathbf{y}[k]$ 可以表示为：

$$\mathbf{y}[k] = \mathbf{s}[k]e^{j(\theta[k]+k2\pi\Delta fT_s)} + \boldsymbol{\eta}[k]$$

$\mathbf{s}[k]$ ：被传输的信号

$\theta[k]$ ：相位噪声

Δf ：载波频率偏移

T_s ：码元

$\boldsymbol{\eta}[k]$ ：加性高斯白噪声

(additive white Gaussian noise ,AWGN)

Section 1

频率恢复

一种有效的频域频率恢复算法是基于对接收信号提高到 **4 倍频的频谱分析**，该算法可扩展到 **M-QAM** 的调制格式，且性能损失可容忍。

频率恢复

假定相位噪声和加性噪声可忽略，且 $\mathbf{s}[k]$ 采用 QPSK 调制，则均衡后信号的计算公式为

$$\mathbf{y}[k] = e^{j(\frac{\pi}{4} + m[k]\frac{\pi}{2})} e^{jk2\pi\Delta f T_s}$$

$m[k] = 0, 1, 2, 3$: QPSK信号的象限

将 $\mathbf{y}[k]$ 信号提高到四次方

$$(\mathbf{y}[k])^4 = e^{j\pi} e^{jk2\pi(4\Delta f)T_s}$$

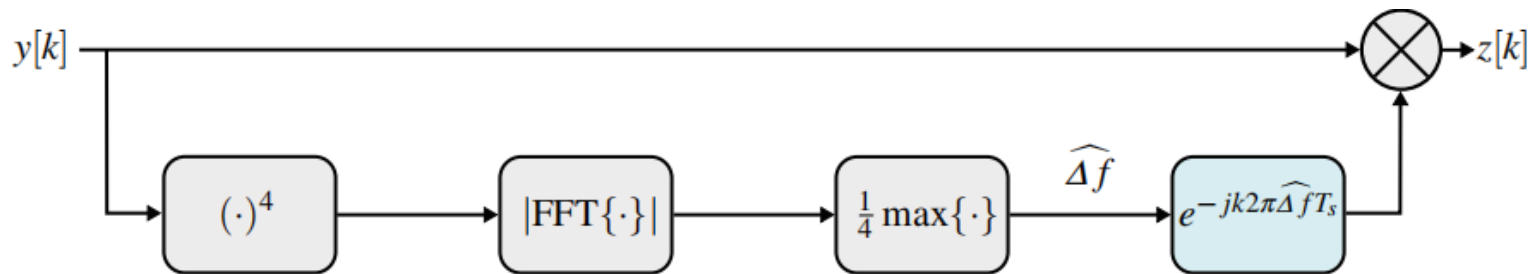
然后，可以通过计算频率并除以 4 来估算频率偏移量

$$\widehat{\Delta f} = \frac{1}{4} \max_f |FFT\{(\mathbf{y}[k])^4\}|$$

载波频率偏移可以被补偿得到

$$\mathbf{z}[k] = \mathbf{y}[k] e^{-jk2\pi\widehat{\Delta f}T_s}$$

频率恢复



频域频率恢复算法：

1. 首先将均衡信号 $y[k]$ 提升到 **4** 倍频，以消除调制信息。
2. 估计的频率偏移 $\widehat{\Delta f}$ 取为最大振幅频谱对应频偏的 **1/4**。
3. 最后，利用 $\widehat{\Delta f}$ 消除频率偏移。

相位恢复

相位噪声 $\theta[k]$ 满足

$$\begin{aligned}\theta[k] &= \theta[k-1] + \Delta\theta[k] \\ \sigma_{\Delta\theta}^2 &= 2\pi\Delta\nu T_s,\end{aligned}$$

$\Delta\theta[k]$: 均值为0的**高斯分布的随机变量**。

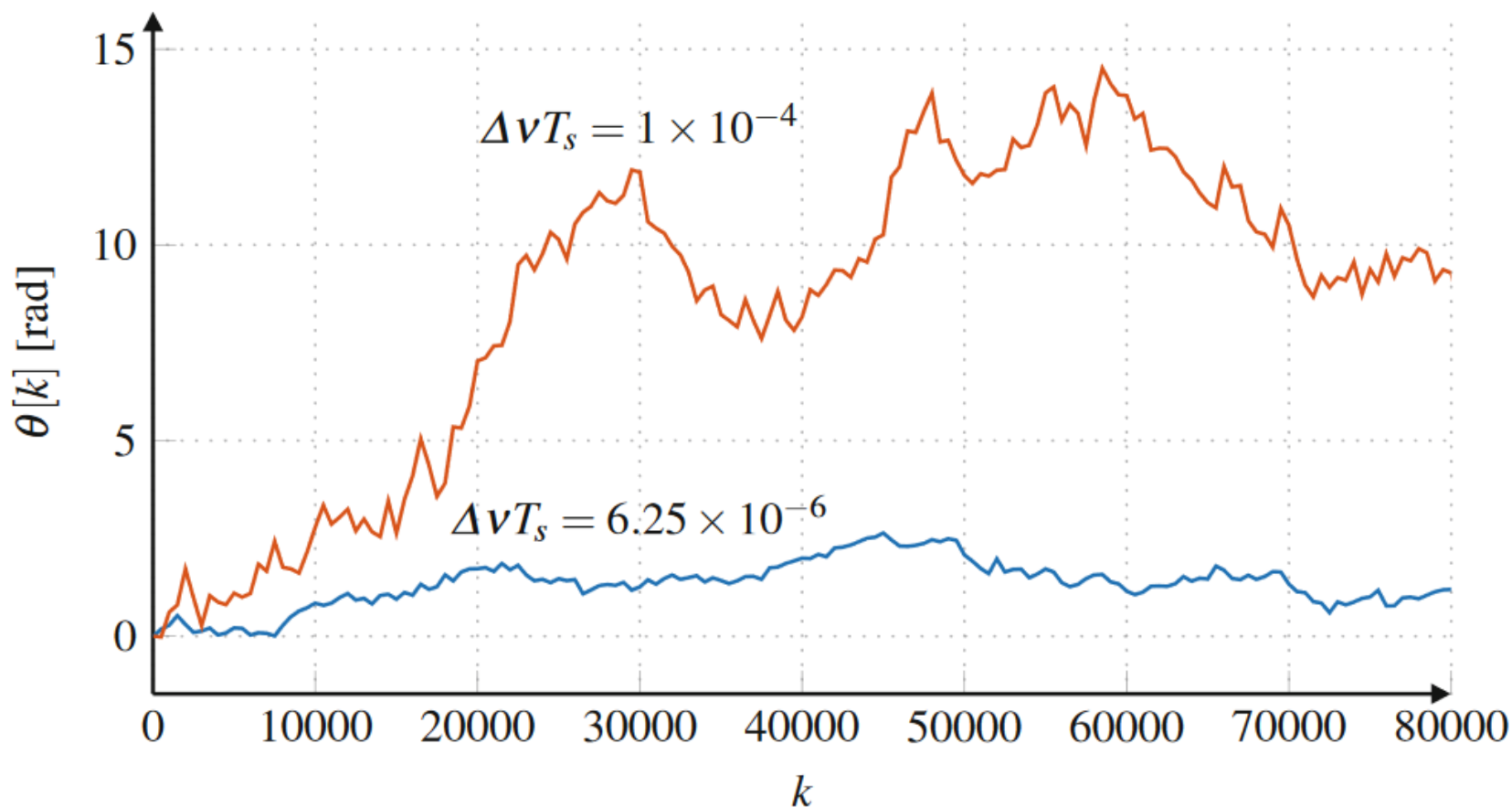
$\sigma_{\Delta\theta}^2$: $\Delta\theta[k]$ 的方差

$\Delta\nu$: **发端和本地激光器的线宽和**

相位噪声强度与激光器线宽成正比，与码元速率成反比。
。

$\Delta\nu T_s$: 用于量化相位噪声对数字相干光通信系统影响程度的性能指标。

相位恢复



不同 $\Delta v T_s$ 激光器的相位噪声实测值。

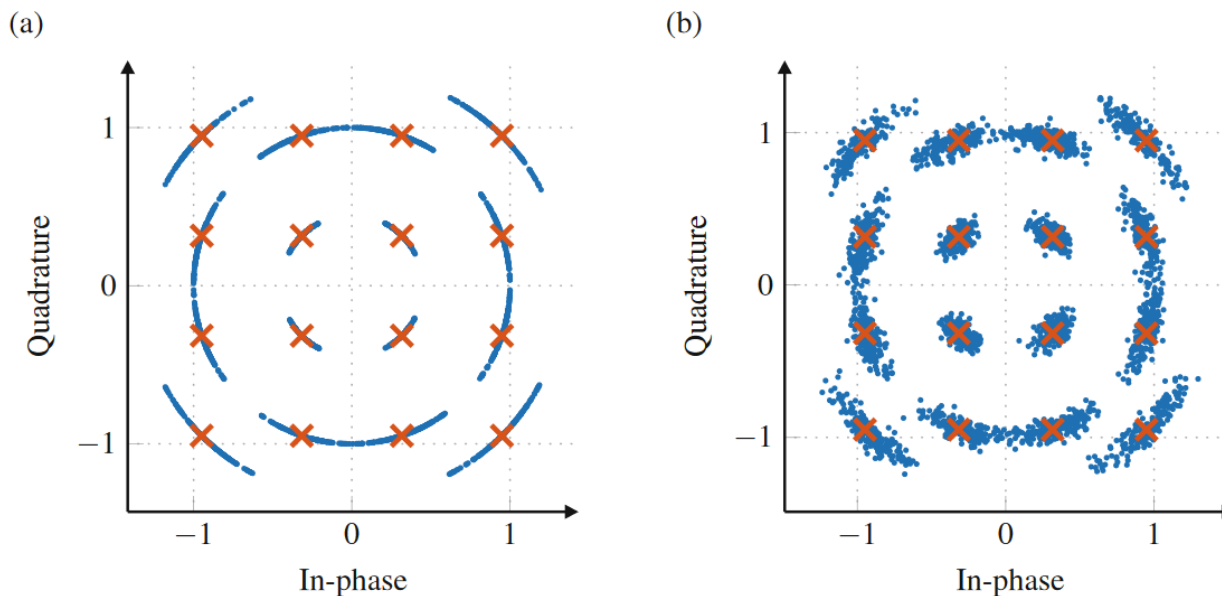
$\Delta v T_s = 1 \text{ e-}4$ 对应20 Gbaud系统中 1 MHz 的激光器, ($\Delta v = 2 \text{ MHz}$) 。

$\Delta v T_s = 6.25\text{e-}6$ 对应 32 Gbaud系统中100kHz 的激光器($\Delta v = 200 \text{ kHz}$)。

相位恢复 思考

传输的符号为 $\mathbf{s}[k]$ ，接收到的符号 $\mathbf{z}[k]$ 在相位噪声 $\boldsymbol{\theta}[k]$ 的损伤下变为

$$\mathbf{z}[k] = \mathbf{s}[k]e^{j\theta[k]} + \boldsymbol{\eta}[k]$$



受相位噪声干扰的80000个码元的16-QAM 星座图， $\Delta\nu T_s = 6.25 \text{ e-6}$.

(a) 无加性噪声. (b) 有加性噪声

相位解包裹

相位噪声可以由维纳随机过程建模，该过程是无约束的。

然而，几种相位恢复算法的输出是一个从 $-\pi/P$ 到 π/P rad 的受限随机过程，其中 P 是一个正整数。在这种情况下，估计的相位噪声过程表现出不连续性，导致相位估计错误。

这种不连续性可以通过相位解包裹来避免，它是一种运算符，用于检测相位演变中的突然跳跃，并通过添加或减去 $2\pi/P$ 的倍数来消除它们。

相位解包裹

一个计算相位解包裹 $\hat{\theta}_{PU}[k]$ 的典型方法是

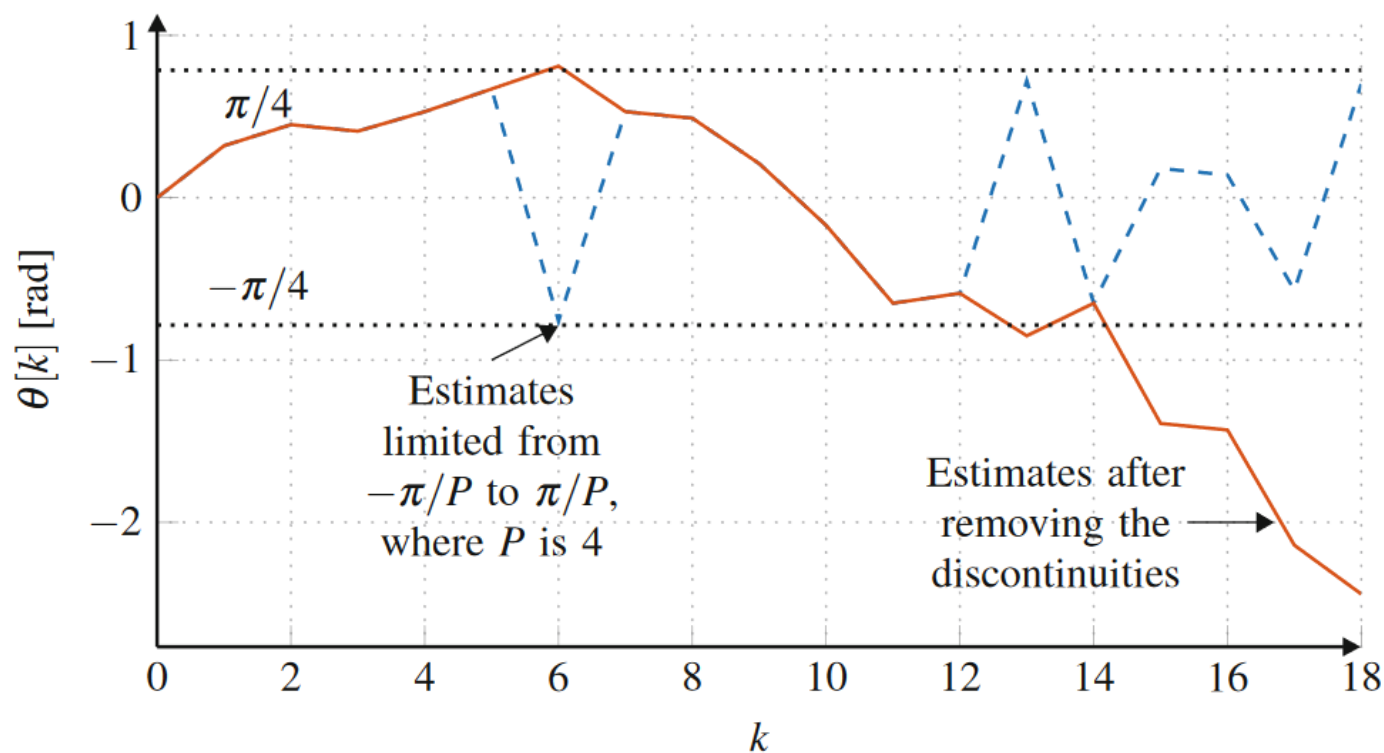
$$\hat{\theta}_{PU}[k] = \hat{\theta}[k] + n[k] \frac{2\pi}{P}$$

$\hat{\theta}[k]$: 相位恢复算法估计的相噪

$n[k]$: 一个满足下式的整数

$$n[k] = \left\lfloor \frac{1}{2} + \frac{\hat{\theta}_{PU}[k-1] - \hat{\theta}[k]}{2\pi/P} \right\rfloor$$

相位解包裹



解包裹之前的估计相位噪声（蓝色虚线）和解包裹之后的估计相位噪声（红色实线）。

该示例假设 $P = 4$ ，这是用于方形M-QAM星座的值。

Viterbi & Viterbi 算法

M-PSK信号 $\mathbf{s}[k]$ 满足

$$s[k] = \sqrt{E_s} e^{j2m[k]\pi/M + j\pi/M}, m[k] \in 0, 1, \dots, M-1,$$

E_s : 码元能量.

计算信号 $\mathbf{s}[k]$ 的**M**次幂

$$s^M[k] = E_s^{M/2} e^{j2m[k]\pi + j\pi}$$

$$s^M[k] = E_s^{M/2} e^{j\pi}$$

M次幂运算消除了 $\mathbf{s}[k]$ 的调制相关性。

Viterbi & Viterbi 算法

在均值为0，方差为 σ_n^2 的加性高斯白噪声 $\eta[k]$ 的干扰下，，且存在乘性维纳相位噪声 $\theta[k]$ 的情况下，接收到的信号 $\mathbf{z}[k]$ ，在提升到M次幂后

$$\begin{aligned} \mathbf{z}^M[k] &= \left(s[k] e^{j\theta[k]} + \eta[k] \right)^M, \\ &\approx E_s^{M/2} e^{j\pi} e^{jM\theta[k]} + \zeta[k], \end{aligned}$$

$\zeta[k]$: 均值为0的加性高斯白噪声，
方差为 $\sigma_\zeta^2 = M^2 E_s^{M-1} \sigma_n^2$

Viterbi & Viterbi 算法

在Viterbi & 算法中，使用一组码元对 $\theta[k]$ 进行估计，滤除 $\zeta[k]$ 的影响

$$\mathbf{z} = [z^M[k - N] \cdots z^M[k - 1] z^M[k] z^M[k + 1] \cdots z^M[k + N]]^T$$

估计 $\theta[k]$ 涉及到当前接收到的符号 $\mathbf{z}[k]$ ，以及 N 个过去的符号和 N 个未来的符号，构成了一个长度为 $L = 2N + 1$ 的滤波器。可以证明，给定 $\mathbf{z}$ ， $\theta[k]$ 的最大似然（ML）估计计算如下：

Viterbi & Viterbi 算法

可以证明，对给定的 $\mathbf{z}$ ， $\boldsymbol{\theta}[k]$ 的最大似然估计计算如下：

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{\text{ML}}[k] &= \max_{\boldsymbol{\theta}[k]} [f_{\mathbf{z}|\boldsymbol{\theta}[k]}(\mathbf{z} | \boldsymbol{\theta}[k])], \\ &= \frac{1}{M} \arg(w_{\text{ML}}^T \cdot \mathbf{z}) - \frac{\pi}{M}\end{aligned}$$

$f_{\mathbf{z}|\boldsymbol{\theta}[k]}(\mathbf{z} | \boldsymbol{\theta}[k])$: 在给定 $\boldsymbol{\theta}[k]$ 的条件下， $\mathbf{z}$ 的概率密度函数

最大似然滤波器的系数 $\mathbf{w}_{\text{ML}} = (\mathbf{1}^T \mathbf{C}^{-1})^{-1}$ 满足 $\mathbf{w}_{\text{ML}}^T \mathbf{C}^{-1} \mathbf{w}_{\text{ML}} = 1$ ，

$\mathbf{1}^T$ ：一个所有元素为1的列向量

Viterbi & Viterbi 算法

协方差矩阵**C**满足：

$$\mathbf{C} = E_s^M M^2 \sigma_{\Delta\theta}^2 \mathbf{K}_{L \times L} + E_s^{M-1} M^2 \sigma_{\eta}^2 \mathbf{I}_{L \times L},$$

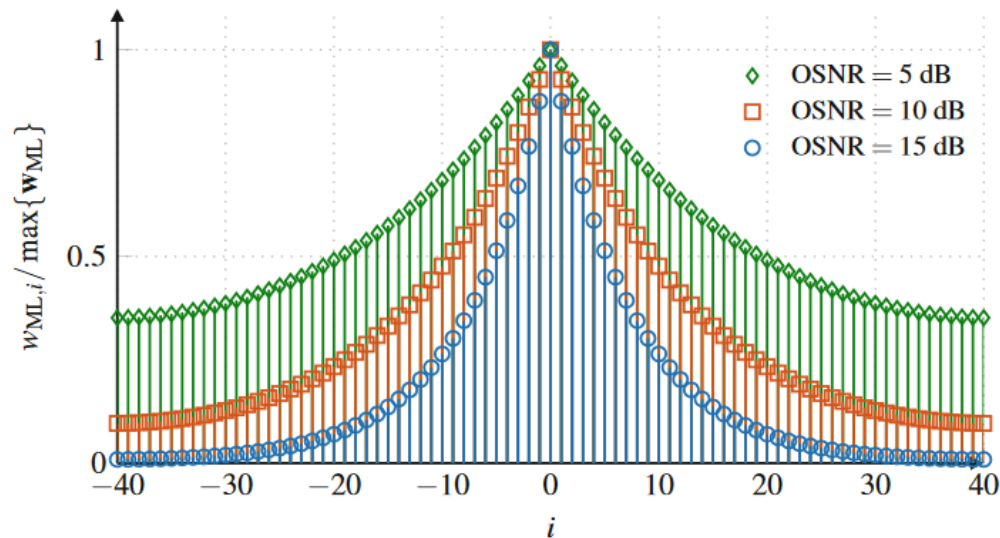
$\mathbf{I}_{L \times L}$: 大小为L的单位阵

$\mathbf{K}_{L \times L}$ 满足：

$$\mathbf{K} = \begin{bmatrix} N & \dots & 2 & 1 & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 2 & \dots & 2 & 1 & 0 & 0 & 0 & \dots & 0 \\ 1 & \dots & 1 & 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & 1 & 1 & \dots & 1 \\ 0 & \dots & 0 & 0 & 0 & 1 & 2 & \dots & 2 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & 1 & 2 & \dots & N \end{bmatrix}.$$

Viterbi & Viterbi 算法

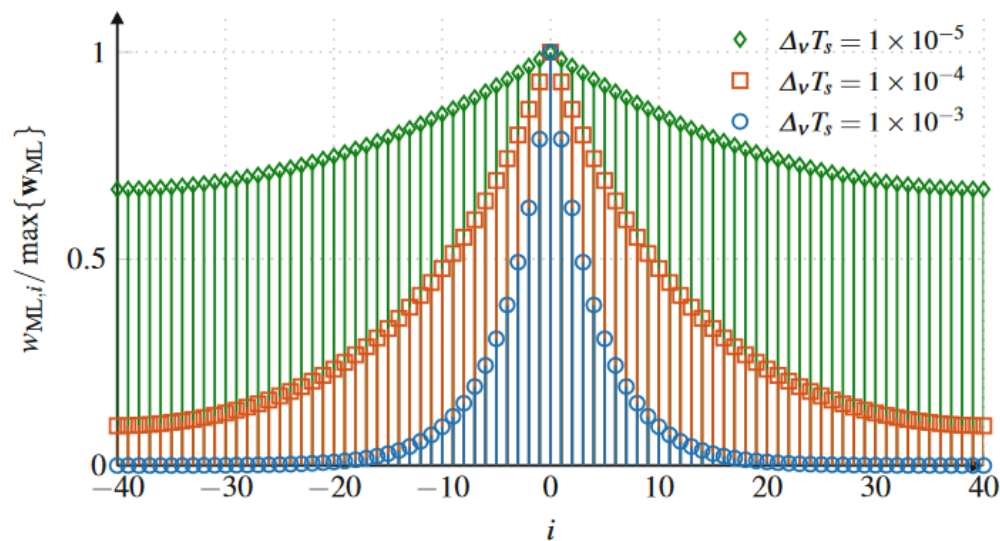
(a)



ML滤波器用于Viterbi & Viterbi算法中相位噪声估计的系数，其中 $L = 81$ 。

(a) 常数 $\Delta v T_S = 1e-4$ 和变化的OSNR。

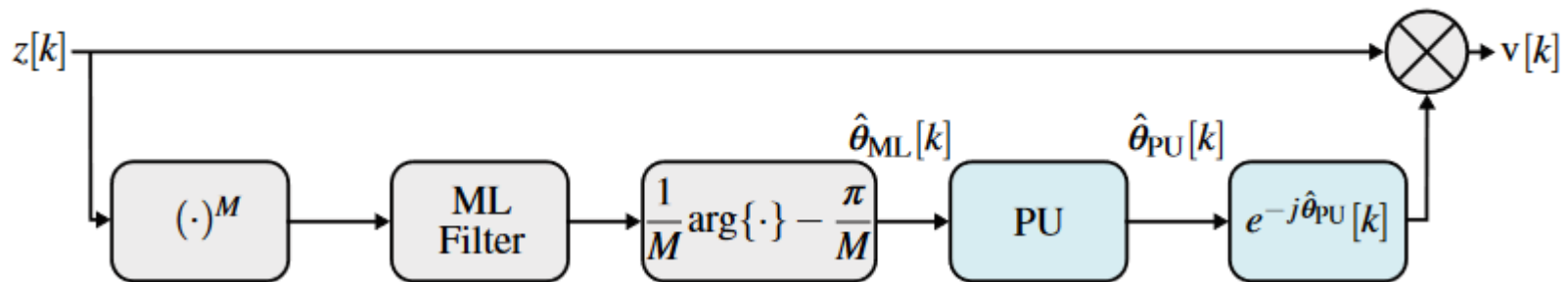
(b)



(b) 常数OSNR = 10 dB和变化的 $\Delta v T_S$ 。

OSNR越低，滤波器越平坦。
相位噪声越大，滤波器的边缘越陡峭。

Viterbi & Viterbi 算法



Viterbi & Viterbi算法的框图：

1. 首先，接收信号向量被提高到**M**次幂，并与**ML**滤波器系数的转置相乘。
2. 然后，算法计算结果值的参数，将其除以**M**，并减去 π/M 。
3. 通过将估计的相位发送到**PU**（相位解包裹）来避免不连续性。

Decision-Directed Algorithm

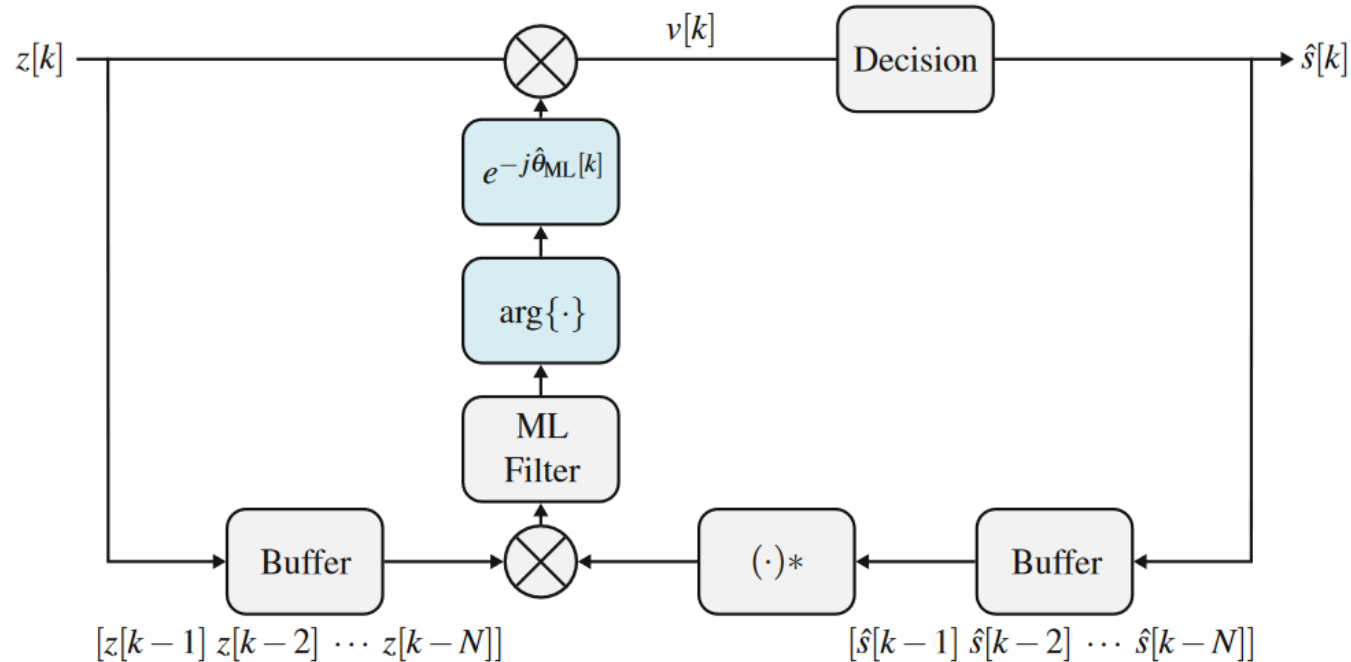
判决引导算法

缺点：

1. 该算法需要使用多个先前的决策来估计当前的相位，这可能对硬件并行性构成问题。
2. 该算法基于先前对相位噪声的估计，引入了在前馈解决方案中不存在的预测误差。

Decision-Directed Algorithm

判决引导算法



DD相位恢复算法：该算法使用接收到的符号向量和已判决符号向量来生成估计的相位。过去的相位估计由最大似然滤波器加权得到。

Decision-Directed Algorithm

判决引导算法

该算法使用接收符号向量 $\mathbf{z}$ 和已判决符号向量 $\hat{\mathbf{s}}^*$ 生成估计的相位 $\hat{\theta}_{ML}[k]$

$$\hat{\theta}_{ML}[k] = \arg(\sum_{i=1}^N w_{ML,i} z[k-i] \hat{s}^*[k-i])$$

$w_{ML,i}$: ML滤波器的系数。

Decision-Directed Algorithm

判决引导算法

DD算法的最大似然滤波器系数计算公式如下：

$$\mathbf{w}_{\text{ML}} = (\mathbf{1}^T \mathbf{C}^{-1})^T,$$

协方差矩阵 $\mathbf{C}$ 如下式：

$$\mathbf{C} = E_s^2 \sigma_{\Delta\theta}^2 \mathbf{K}_{N \times N} + E_s \sigma_{\eta}^2 \mathbf{I}_{N \times N}$$

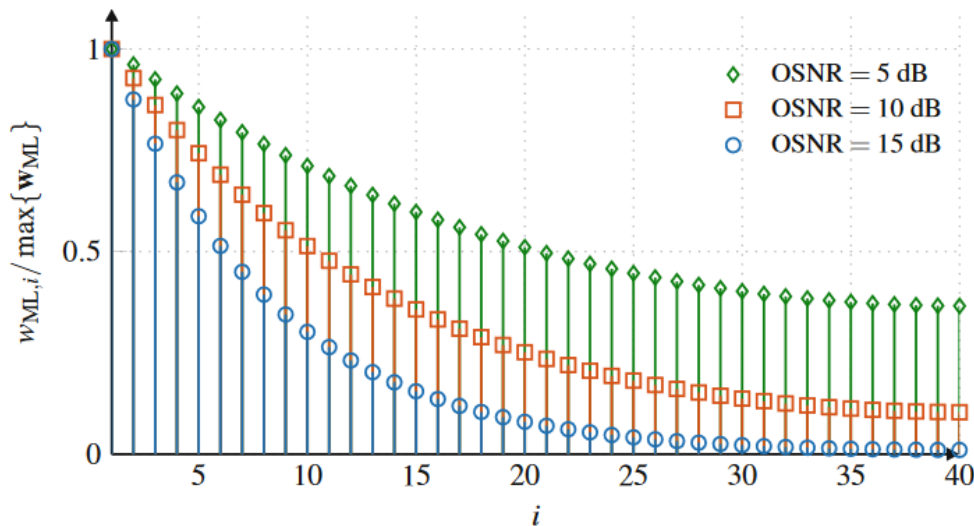
$\mathbf{K}_{L \times L}$ 如下式：

$$\mathbf{K} = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 1 & \cdots & 1 \\ 0 & 1 & 2 & \cdots & 2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & 2 & \cdots & N-1 \end{bmatrix}.$$

Decision-Directed Algorithm

判决引导算法

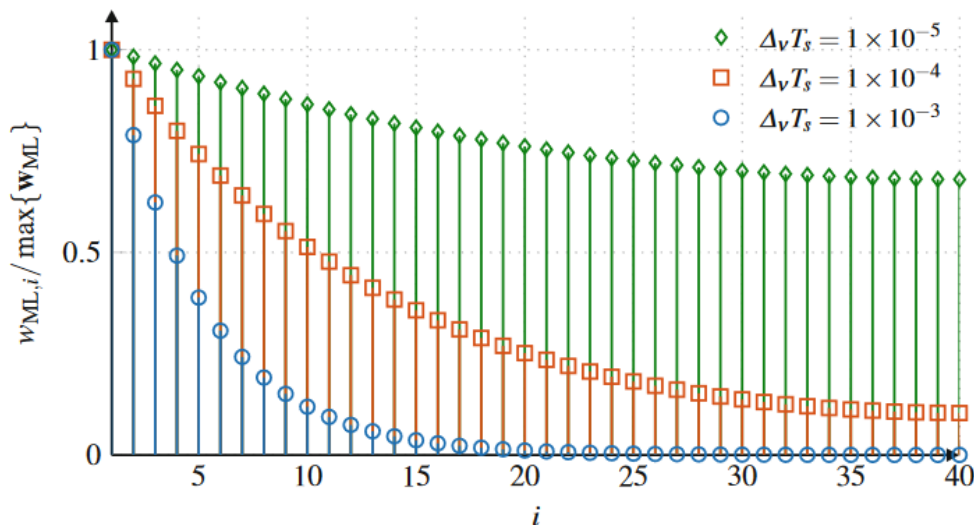
(a)



在DD算法中用于相位噪声估计的ML滤波器系数，其中 $N = 40$ 。

(a) 以常数 $\Delta v T_s = 1e-4$ 和变化的OSNR。

(b)



(b) 以常数OSNR = 10 dB和变化的 $\Delta v T_s$ 。

OSNR越低，滤波器越平坦，因为需要更多的符号来滤除附加噪声。

相位噪声越高，滤波器的边缘越陡峭，因为相位噪声样本的相关性较低。

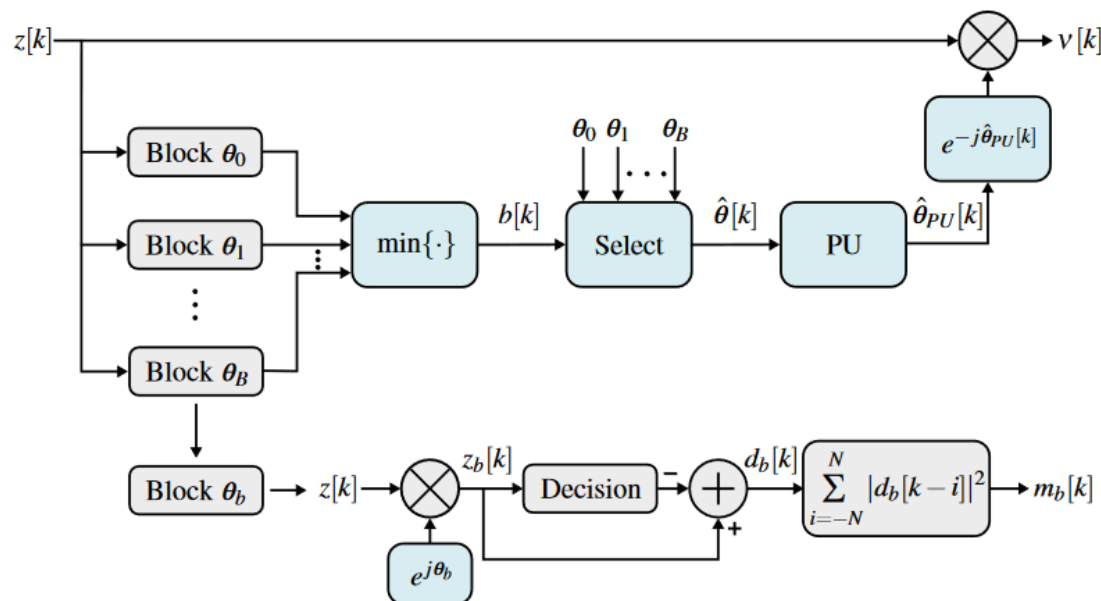
The Blind Phase Search Algorithm

盲相位搜索算法

DD相位恢复方案对于**M-QAM**星座显示出良好的性能，但**反馈环路**可能会在实际应用的特定集成电路（**ASIC**）部署中存在**实际应用的问题**，考虑到所需的高度并行性。
盲相位搜索（BPS）算法通过适用于并行实现的前馈架构，在高阶**M-QAM**调制格式下具备出色的性能。

The Blind Phase Search Algorithm

盲相位搜索算法



BPS算法:

1. 输入信号 $\mathbf{z}[k]$ 在 $\mathbf{B}$ 个测试相位下旋转。
2. 随后，对旋转后的符号进行最小距离运算，并计算判决操作前后符号之间的平方距离。
3. 为了减轻加性噪声的影响，通过相同测试相位旋转的 $2N + 1$ 个连续符号的平方距离被相加，产生信号 $m_b[k]$ 。
4. 估计的相位 $\hat{\theta}[k]$ 被选为最小化 $m_b[k]$ 的 $\theta_b[k]$ 。需要使用 **PU** 来避免不连续性

The Blind Phase Search Algorithm

盲相位搜索算法

将接收信号进行 B 个相位的旋转测试，每个相位 θ_b 为：

$$\theta_b = \frac{b}{B}p, b \in \{-\frac{B}{2}, \dots, -1, 0, 1, \dots, \frac{B}{2} - 1\}$$

旋转后的符号 $z_b[k]$ 为：

$$z_b[k] = z[k]e^{j\theta_b}$$

旋转的范围 p 等于信号星座的不确定角度。对于 **M-QAM** 格式， $p = \pi/2$ 。

The Blind Phase Search Algorithm

盲相位搜索算法

旋转后的符号 $z_b[k]$ 随后进行最小距离判决运算，计算判决前后符号的平方距离 $|d_b[k]|^2$

$$|d_b[k]|^2 = |z_b[k] - [z_b[k]]_D|^2$$

$[\cdot]_D$: 最小距离判决运算

The Blind Phase Search Algorithm

盲相位搜索算法

为减轻加性噪声对相位恢复性能受到的影响，将由同一测试相位旋转的 $2N + 1$ 个连续符号的二次方距离相加

$$m_b[k] = \sum_{i=-N}^N |d_b[k - i]|^2$$

!!!!，N 的选择取决于相位噪声功率和 OSNR 的权衡。

The Blind Phase Search Algorithm

盲相位搜索算法

最后，估计相位 $\hat{\theta}[k]$ 被选为使 $\theta_{b[k]}$ 最小化的 $m_b[k]$

$$b[k] = \min_b \{m_b[k]\}$$

$$\hat{\theta}[k] = \theta_{b[k]}$$

估计相位 $\hat{\theta}[k]$ 的跨度限于 π 的弧度。估计相位 $\hat{\theta}[k]$ 在 $-\pi/2 \sim \pi/2$ rad之存在有偏移。

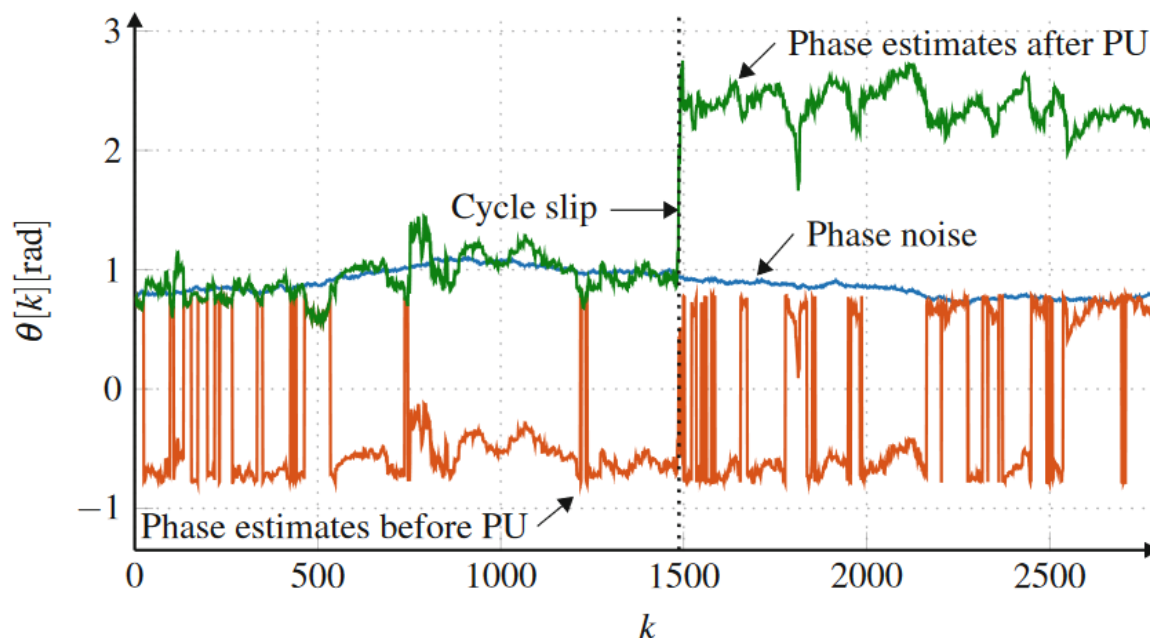
因此，需要使用 **PU**（相位解包裹）来避免相位不连续。

Cycle Slips 周跳

为了允许相位估计值从 $-\infty$ 到 $+\infty$ ，相位恢复算法采用了相位解包裹（**PU**）。

PU 操作虽然必要，但也容易出错。在相加噪声和相位噪声较高的情况下，**PU** 可能会添加或减去不正确的 $\pi/2$ 倍数，从而导致称为周跳（**CS**）的星座旋转。未补偿的 **CS** 可产生灾难性的误码。

Cycle Slips 周跳



维纳相位噪声（蓝线）及其在 **PU** 操作前（橙线）和**PU**操作后（绿线）的估计值。假设采用方波 **QAM** 格式，**PU** 操作前的相位估计值限制在 $-\pi/4$ 和 $\pi/4$ 之间。

PU 操作是允许相位估计值偏离范围在 $-\infty$ 和 ∞ 之间，从而遵循原始相位噪声过程轨迹的必要条件。起初，**PU** 通过在负相位估计值上添加 $\pi/2$ 来成功避免相位反弹。然而，由于 **AWGN** 很大，它错误地将 $\pi/2$ 添加到了正相位的样本中，导致周跳。

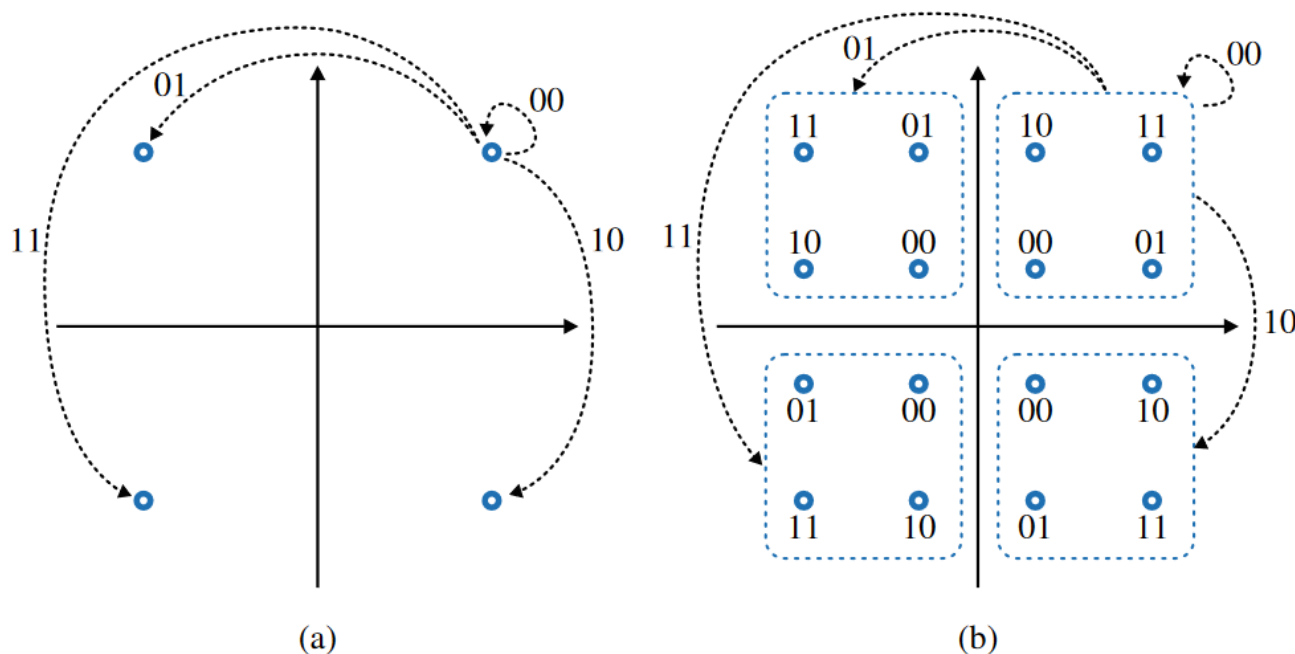
Differential Encoding 差分编码

减少周跳的措施：

- 前向纠错（**FEC**）解码器。
- 定期插入先导符号：周跳导致的错误脉冲串被限制在导频序列之间，可通过后续的 **FEC** 方案消除。
- 差分编码（**DE**）和解码：避免周跳造成的错误传播的最常用技术。

Differential Encoding 差分编码

差分编码（**Differential Encoding, DE**）的原理是将两个信息比特编码在连续码元间的差分象限中。



对于**QPSK**，两个比特定义了连续符号之间的象限变化。比特**00**、**01**、**10**和**11**分别表示**0**、 $\pi/2$ 、 $3\pi/2$ 和 π 弧度的相位变化。对于**16-QAM**，四位中的两位定义了象限之间的变化，而另外两位在一个象限内进行格雷映射。

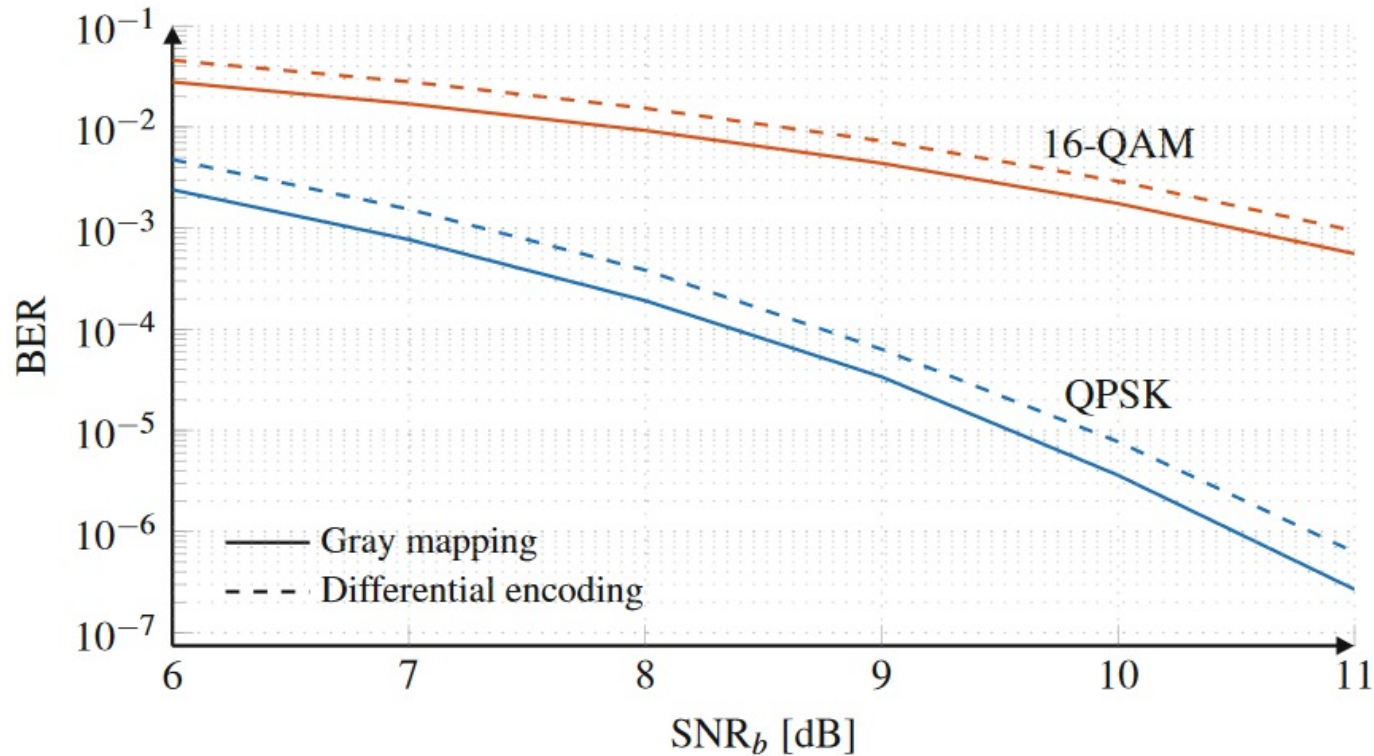
Differential Encoding 差分编码

一次周跳会导致一个码元错误，但错误不会进一步传播。

尽管差分编码（**DE**）简单有效，但在没有循环滑移的情况下会增加比特错误率（**BER**）。

加性噪声可能导致错误的象限决策，在差分解码后生成两个连续的符号错误。

Differential Encoding 差分编码



QPSK和16-QAM格式下不同信噪比的BER，假设为AWGN信道

附录

Frequency recovery algorithm 频率恢复

```
% Obtaining the absolute value of the spectrum of the signal^4:  
SignalSpectrum = fftshift(abs(fft(y(:,1).^4)));  
  
% Obtaining the frequency offset:  
Delta_f = (1/4)*f(SignalSpectrum == max(SignalSpectrum));
```

$$\widehat{\Delta f} = \frac{1}{4} \max_f |FFT\{(y[k])^4\}|$$

附录

ML filter for Viterbi & Viterbi algorithm

```
% Obtaining the covariance matrix:  
C = Es^M*M^2*Sigma_DeltaTheta2*K + Es^(M-1)*M^2*Sigma_eta2*I;  
  
% Filter coefficients:  
wML = (ones(L,1)'/(C)).' ; wML = wML/max(wML);
```

$$C = E_s^M M^2 \sigma_{\Delta\theta}^2 K_{L \times L} + E_s^{M-1} M^2 \sigma_{\eta}^2 I_{L \times L}$$

$$w_{ML} = (\mathbf{1}^T C^{-1})^T$$

附录

Viterbi & Viterbi algorithm

`% generate phase estimates:`

```
ThetaML(:,Pol) = (1/M)*angle(wML.'*(zBlocks.^M))-pi/M;
```

$$\begin{aligned}\hat{\theta}_{\text{ML}}[k] &= \max_{\theta[k]} [f_{z|\theta[k]}(z | \theta[k])] \\ &= \frac{1}{M} \arg(w_{\text{ML}}^T \cdot z) - \frac{\pi}{M}\end{aligned}$$

`% Phase unwrapping:`

```
for i = 1:size(ThetaML,1)
```

`% Phase unwrapper:`

```
n = floor(1/2 + (ThetaPrev - ThetaML(i,:))/(2*pi/M));
```

```
ThetaPU(i,:) = ThetaML(i,:) + n*(2*pi/M); ThetaPrev = ThetaPU(i,:);
```

```
end
```

$$n[k] = \left\lfloor \frac{1}{2} + \frac{\hat{\theta}_{PU}[k-1] - \hat{\theta}[k]}{2\pi/P} \right\rfloor \quad \hat{\theta}_{PU}[k] = \hat{\theta}[k] + n[k] \frac{2\pi}{P}$$

附录

DD phase recovery algorithm 直检相位恢复

```
% Phase noise estimates:
for i = 1:length(z)
    % Obtaining the phase of the previous received symbol:
    Theta(i,:) = angle(wML'*b);

    % Compensating for the phase of the received symbols:
    v = z(i,:).*exp(-1i*Theta(i,:));

    % Deciding the symbols after phase recovery:
    s_Decided(i,:) = Decision(v,ModFormat,false);

    % Updating the buffer b = z*s_Decided:
    b = circshift(b,1,1);
    b(1,:) = z(i,:).*conj(s_Decided(i,:))...
        ./abs(z(i,:).*conj(s_Decided(i,:)));
end
```

$$\hat{\theta}_{\text{ML}}[k] = \arg\left(\sum_{i=1}^N w_{\text{ML},i} z[k-i] \hat{s}^*[k-i]\right)$$

附录

BPS algorithm 盲相位搜索

```
% Phase noise estimates:
for i = 1:size(zBlocks,2)
    % Applying the test phase angles to the symbols:
    zRot = repmat(zBlocks(:,i,:),1,B,1).*ThetaTestMatrix;

    % Decision of the rotated symbols:
    zRot_Decided = Decision(zRot,ModFormat,false);

    % Intermediate signal to be minimized:
    m = sum(abs(zRot-zRot_Decided).^2,1);

    % Estimating the phase noise as the angle that minimizes 'm':
    [~,im] = min(m,[],2) ; Theta = reshape(ThetaTest(im),1,NPol);

    % Applying the phase unwrapper to the estimated phase angles:
    ThetaPU(i,:) = Theta + floor(0.5 - (Theta-ThetaPrev)./(p)).*(p);

    % Updating the previous phase variable:
    ThetaPrev = ThetaPU(i,:);
end
```

$$z_b[k] = z[k]e^{j\theta_b}$$

$$m_b[k] = \sum_{i=-N}^N |d_b[k-i]|^2$$

$$b[k] = \min_b \{m_b[k]\}$$

$$\hat{\theta}[k] = \theta_b[k]$$